WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Karnaugh Maps

Lecture 05: simplifying Boolean functions

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Why simplify?

02 The Karnaugh map

03 Grouping

04 Four variables

05 Don’t cares and products of sums

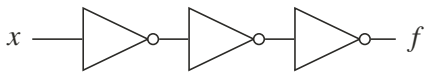
06 Summary

01

Why simplify?

Fewer gates

The circuit below computes $f(x) = x'''$:



Since $x'' = x$ (involution), $f(x) = x'$: one NOT gate does the same job.

- Both circuits give the same output, but the long one needs three gates instead of one: it costs more, and the signal is slower to pass through it.
- Fewer and smaller gates mean cheaper and faster circuits. Minimising Boolean functions is worth the effort.

Canonical forms are long

- A literal is a variable or its complement: x, x', …
- From Lecture 04: $\sum m(1,3,5,6,7) = x'y'z + x'yz + xy'z + xyz' + xyz$ has 5 products and 15 literals.
- The same function is $xy + z$: 2 products and 3 literals.

The laws of Lecture 03 get you there, but you have to see which law to apply next. A Karnaugh map (K-map) turns the search into a picture.

Introduced by Maurice Karnaugh in 1953.

02

The Karnaugh map

Neighbours combine

Two minterms that differ in exactly one variable combine into one shorter product:

$xyz' + xyz = xy(z' + z) = xy \cdot 1 = xy$ distributivity, complement, identity

And two such pairs, differing in one more variable, combine again:

$$x'y'z + x'yz + xy'z + xyz = x'z + xz = z.$$

> **The idea**
>
> Draw the truth table as a grid in which cells next to each other differ in exactly one input bit. Then the products that combine are the cells that sit together.

Gray code

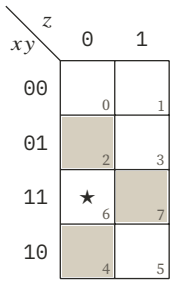
To order the rows so that neighbours differ in one bit, do not count in binary:

binary order	00	01	10	11	
bits that change		1	2	1	2 back to 00
Gray code	00	01	11	10	
bits that change		1	1	1	1 back to 00

> **Definition (Gray code)**
>
> A *Gray code* lists the n-bit numbers so that each one differs from the next in a single bit. For two bits: 00, 01, 11, 10.

The last code also differs from the first in one bit: the order wraps around.

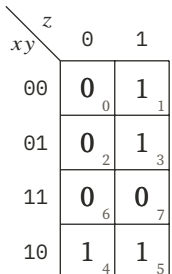
A map of three variables



- Rows: xy in Gray-code order. Columns: z.
- Each cell is one row of the truth table; the small number is its minterm number.
- The cell ★ is input 110. Its neighbours, shaded, are 010, 100 and 111: each differs from 110 in one bit.
- The map wraps around: the top row (00) and the bottom row (10) are neighbours too.

Filling in the map

Simplify $f(x, y, z) = (x + y + z)(x + y' + z)(x' + y' + z)(x' + y' + z')$.



1. The four maxterms are M_0, M_2, M_6 and M_7: write a 0 in those cells.
2. Every other cell is a 1: $f = \sum m(1, 3, 4, 5)$.

From a truth table, copy each row's output into its cell. Mind the order of the rows: 00, 01, 11, 10.

03

Grouping

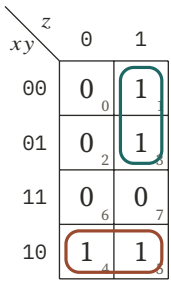
Groups

Rules for groups

- A group contains only 1s.
- It is a rectangle of 1, 2, 4, 8, … cells: a power of two.
- It may wrap around the edges of the map.
- Groups may overlap.

- Make every group as large as possible, and use as few groups as possible to cover all the 1s.
- Each group becomes one product. Keep the variables that stay the same in all its cells (as they are if 1, complemented if 0) and drop the others.
- In a map of n variables, a group of 2^k cells has $n - k$ literals.

Reading the groups

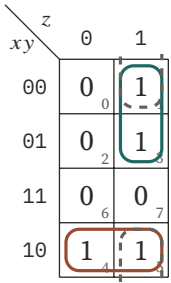


- Teal, cells 001 and 011: $x = 0$ and $z = 1$ stay the same, y changes. Product: $x'z$.
- Brick, cells 100 and 101: $x = 1$ and $y = 0$ stay the same, z changes. Product: xy'.

$$f(x, y, z) = xy' + x'z$$

4 products and 12 literals became 2 and 4.

Prime and essential prime implicants



Definition (Prime implicant)

A group of 1s that cannot be made any larger.

Definition (Essential prime implicant)

A prime implicant that covers a 1 which no other prime implicant covers.

The dashed group, cells 001 and 101 (wrapping around), is $y'z$. It is prime, but not essential: both its 1s are covered already.

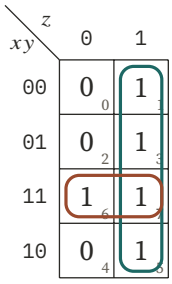
Coming back: $xy' + x'z + y'z = xy' + x'z$ is the consensus theorem of Lecture 03, and $y'z$ is the consensus term.

The procedure

1. Fill in the map from the truth table or the canonical form.
2. Find the prime implicants: the largest groups of 1s.
3. Take every essential prime implicant.
4. If any 1s are still uncovered, cover them with as few further prime implicants as possible.
5. Write one product per group, and add them up.

> **Remark**
>
> The result is a minimal sum of products. It need not be unique: sometimes two different choices in step 4 are equally small.

Back to Lecture 04



$f = \sum m(1, 3, 5, 6, 7)$.

- Teal: the whole column $z = 1$, four cells. Only z stays the same: z.
- Brick: row 11, two cells: xy. It overlaps the teal group, and that is fine.

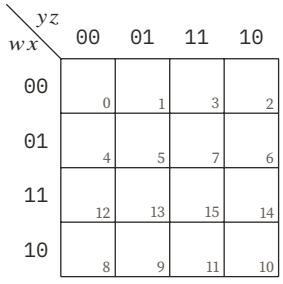
$$f(x, y, z) = xy + z$$

The expression we started from in Lecture 04.

04

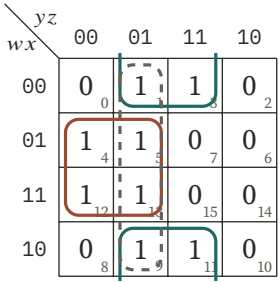
Four variables

A map of four variables



- Rows: wx; columns: yz; both in Gray-code order.
- Cell number = 4× row value + column value: row 11, column 01 is m_{13}.
- The map wraps around both ways: top and bottom rows are neighbours, and so are the left and right columns.
- So the four corners are neighbours, and can form one group.

Example: wrap-around



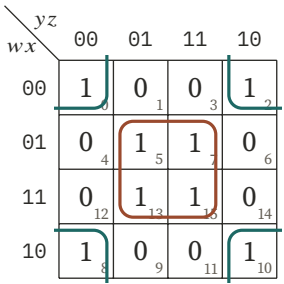
$f = \sum m(1, 3, 4, 5, 9, 11, 12, 13)$.

- Teal, wrapping top to bottom: $x = 0$, $z = 1$. Product: $x'z$.
- Brick: $x = 1$, $y = 0$. Product: xy'.
- Dashed: the column $yz = 01$ is $y'z$, prime but not essential.

$$f(w, x, y, z) = x'z + xy'$$

Each group has 4 cells: $4 - 2 = 2$ literals.

Example: the four corners



$f = \sum m(0, 2, 5, 7, 8, 10, 13, 15)$.

- Teal, the four corners: $x = 0$, $z = 0$. Product: $x'z'$.
- Brick, the centre: $x = 1$, $z = 1$. Product: xz.

$$f(w, x, y, z) = x'z' + xz$$

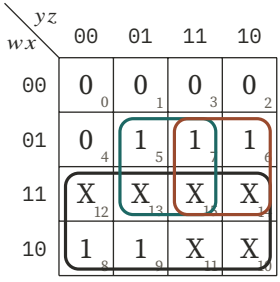
f is 1 when x and z are equal; w and y do not matter.

05

Don’t cares and products of sums

Don’t-care conditions

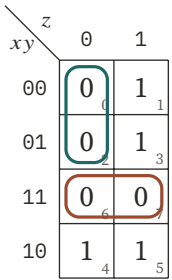
Sometimes some inputs never occur, or their output does not matter. Mark them X: each X may be grouped as a 1 or left out, whichever makes the groups larger.



The input $wxyz$ is a decimal digit, 0 to 9, in binary; inputs 10 to 15 never occur. f is 1 when the digit is 5 or more.

- With the X’s: $f = w + xz + xy$.
- Without them: $f = w'xz + w'xy + wx'y'$, three products of three literals.

Products of sums from a map



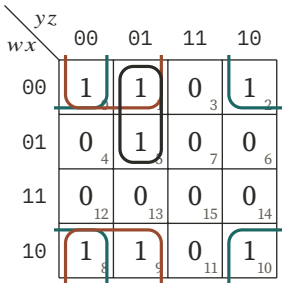
Group the 0s instead. Each group becomes one sum: keep the variables that stay the same, as they are if 0 and complemented if 1.

- Teal, cells 000 and 010: $x = 0$, $z = 0$. Sum: $x + z$.
- Brick, cells 110 and 111: $x = 1$, $y = 1$. Sum: $x' + y'$.

$$f = (x + z)(x' + y')$$

Multiply out: $xy' + x'z + y'z$, and the consensus term $y'z$ drops: the same $f = xy' + x'z$ as before.

Try it



1. Minimise $f = \sum m(0, 1, 2, 5, 8, 9, 10)$.
   $f = x'z' + x'y' + w'y'z$: the corners, a wrapped group of four, and a pair.
2. Minimise the majority function of Lecture 04, $\sum m(3, 5, 6, 7)$.
   $xy + xz + yz$: three pairs, all essential.

06

Summary

Summary

- Minimal expressions mean fewer gates: cheaper and faster circuits.
- A K-map is a truth table arranged in Gray-code order, so that neighbouring cells differ in one bit. It wraps around at the edges.
- Group the 1s in rectangles of 2^k cells, as large and as few as possible; each group is one product of $n - k$ literals.
- Essential prime implicants are always needed; others only to cover what is left.
- Don't-cares (X) may be used as 1s. Grouping the 0s gives a product of sums.
- K-maps work well up to four variables. Beyond that, computers minimise functions, with methods such as Quine–McCluskey.

Before the next lecture

- Read Section 2.3.2 of the book, *Boolean Function Simplification with Karnaugh Maps*.
- Minimise $\sum m(0, 2, 4, 5, 6, 7)$ as a sum of products and as a product of sums.
- Minimise $\sum m(0, 1, 5, 7, 13, 15)$ in four variables, and find a prime implicant you do not need.
- Open Logisim-evolution and find the AND, OR and NOT gates.

> **Next lecture** (Unit 6)
>
> Logic gates: NOT, AND, OR, XOR, NAND, NOR, XNOR, and universal gate sets.